Noise & Restoration

Week 4 — Session 1: Modelling noise & the two core filters

Practical Image Processing

Lecture 4.1 ▪ 60–70 min

Recap & today

- Wk3: filters combine a pixel with its neighbours (convolution).
- Today: **noise** — where it comes from, how to **model** it, and the **two core filters** (averaging & median) that cover most cases.
- Session 2 will add edge-preserving filters, deblurring, and how to *measure* restoration.

Why it matters

Real meter photos are noisy (low light, high ISO). Denoising is what makes the digits thresholdable and readable for DocScan.

Roadmap

Noise models

Averaging & Gaussian smoothing

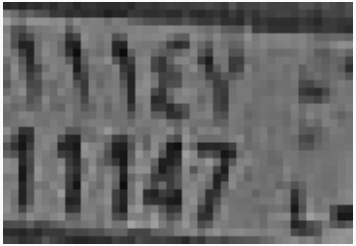
The median filter

In-class exercise

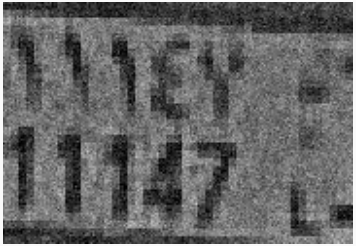
Wrap-up

Noise models

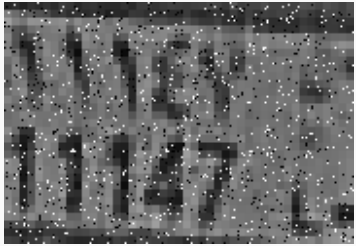
Two noises you must recognize



clean



Gaussian



salt-and-pepper

Key idea

The noise *type* decides the *cure*. Gaussian $\rightarrow$ averaging; impulse $\rightarrow$ median. Diagnose first.

Where noise comes from (physics, Week 1)

- **Gaussian noise**: small random fluctuations added to every pixel (sensor/read noise, amplifier noise).
- **Salt-and-pepper** (impulse): random pixels stuck at 0 or 255 (dead pixels, bit errors in transmission).
- **Poisson** (shot) noise: from photon counting; stronger in dark areas / low light.

Math

Additive Gaussian model: $g(x, y) = f(x, y) + n(x, y)$, with $n \sim \mathcal{N}(0, \sigma^2)$ — additive, zero-mean, and independent of the signal.

Poisson (shot) noise — signal-dependent

- Each pixel counts photons; the count follows a **Poisson** distribution with mean λ.
- Key property: variance *equals* the mean, so $\sigma = \sqrt{\lambda}$ — brighter pixels have more *absolute* noise but less *relative* noise.
- That is why shadows look grainier than highlights in a low-light meter photo.

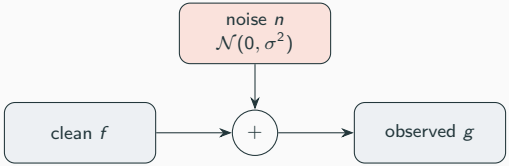
Math

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad \mathrm{Var} = \mathbb{E} = \lambda, \qquad \mathrm{SNR} = \frac{\lambda}{\sqrt{\lambda}} = \sqrt{\lambda}.$$

Note

Collect more light (longer exposure, more photons) and the SNR rises as $\sqrt{\lambda}$ — hardware beats software.

The additive model, visually



- Denoising = estimate f given only g. We never see n directly.
- Because n is **zero-mean**, *averaging* independent samples of the same signal drives n toward 0 — the whole basis of smoothing filters.

Diagnosing noise

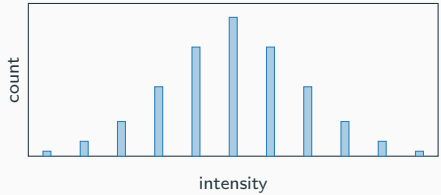
- Look at a **flat** region: Gaussian = a fuzzy spread; salt-pepper = isolated black/white specks.
- The histogram of a flat patch: Gaussian widens the peak into a bell; impulse adds spikes at 0 and 255.
- Match the filter to what you see — using the wrong one wastes detail or fails to clean.

✕ Pitfall

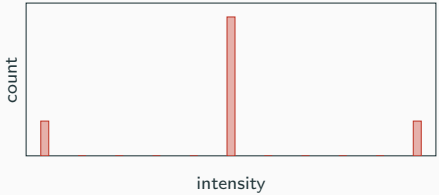
Averaging (Gaussian blur) barely touches salt-and-pepper — it just spreads each speck into a gray smudge.

Histogram of a flat patch

Gaussian: a widened bell



Salt-pepper: peak + two spikes



Key idea

One glance at the flat-patch histogram tells you which family you are fighting — bell vs spikes.

Worked: reading noise off a patch

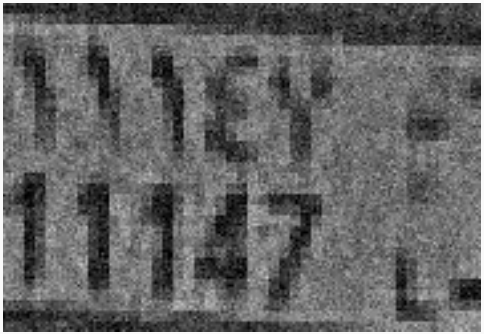
A supposedly uniform gray wall (≈ 120) measured as:

118	122	255	119	121
120	0	121	123	118
119,124,120,255,122				

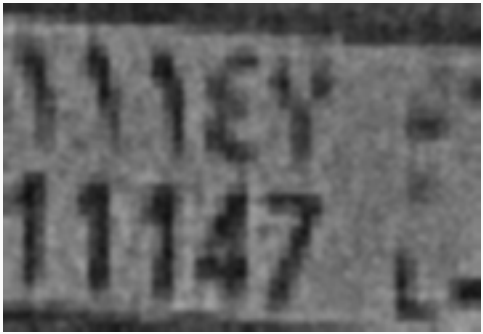
- Most values cluster near 120 with a ± 4 spread $\rightarrow$ a **Gaussian** component.
- Three values are **0** or **255** — nowhere near 120 $\rightarrow$ **impulse** contamination.
- Diagnosis: *both* noises present $\Rightarrow$ median first (kill specks), then a light average.

Averaging & Gaussian smoothing

Removing Gaussian noise by averaging



noisy



Gaussian blur

- Averaging a neighbourhood sums many noisy samples of (nearly) the same signal — the noise partly cancels.
- Cost: real edges also get averaged, so the result is smoother *and* softer. (Session 2 fixes this.)

Why averaging works: the $\sqrt{N}$ rule

Math

Average N independent samples of variance σ^2: $\quad \mathrm{Var}\left(\frac{1}{N}\sum n_i\right) = \dfrac{\sigma^2}{N} \;\Rightarrow\; \sigma_{\text{out}} = \dfrac{\sigma}{\sqrt{N}}.$

- A 3×3 mean averages $N = 9$ pixels $\rightarrow$ noise drops by $\sqrt{9} = 3\times$.
- A 5×5 mean ($N = 25$) $\rightarrow 5\times$ less noise, but $5\times$ more blur too.
- Diminishing returns: to halve noise again you must *quadruple* the window.

Key idea

Bigger window = less noise *and* more blur. The window size *is* the trade-off knob.

Mean filter vs Gaussian filter

box (mean) kernel

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

All neighbours weighted equally → slightly boxy, can ring.

Gaussian kernel ($\sigma \approx 1$)

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Centre-weighted → smoother, no boxiness, separable & fast.

✔ Good practice

Both kernels sum to 1 (normalized) so the average brightness is preserved. Prefer the Gaussian: it is isotropic and separable.

Worked: mean of a 3×3 patch

Noisy patch around a flat gray value:

121	118	122
119	255	120
123	117	120

- **Mean** $= \frac{1}{9}(121+118+122+119+255+120+123+117+120) = \frac{1215}{9} = 135$.
- The single **255** outlier dragged the estimate from ~ 120 up to **135**.
- Averaging *smeared* the impulse into every output near it instead of removing it.

✗ Pitfall

One impulse pixel poisons the whole window under averaging. Remember this when we compare with the median.

Choosing σ and window size

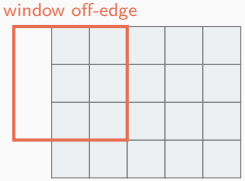
- Rule of thumb: kernel radius $\approx 3\sigma$ so the tail is negligible; e.g. $\sigma = 1 \Rightarrow$ a 7×7 ($r = 3$) window.
- Larger σ = stronger smoothing = more noise gone *and* more detail gone.
- Pick σ from the *feature* you must keep: it should be smaller than the thinnest stroke of a digit.

Note

For meter digits a few pixels wide, $\sigma = 1$ (3×3) is usually the ceiling — beyond it the strokes fuse.

Boundary handling

- The window falls off the image edge — what fills the missing neighbours?
- **Replicate** (clamp) the edge pixel, **reflect** the image, **wrap** around, or **zero**-pad.
- Zero-padding darkens borders (a black frame bleeds in); *reflect* is the safest default.

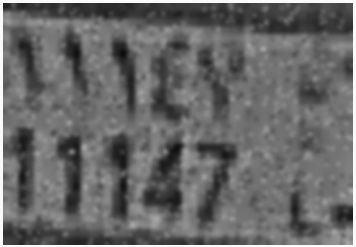


The median filter

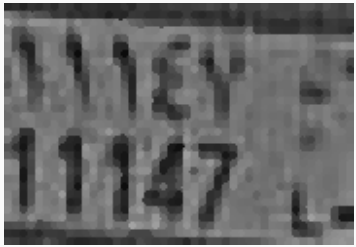
Removing salt-and-pepper — median wins



noisy



Gaussian (bad)



median (great)

Key idea

The median ignores outliers: a speck of 255 among 100s doesn't move the median. This is why median crushes impulse noise while keeping edges.

Why median beats averaging for impulse noise

- Mean of $\{100, 100, 100, 255\} = 164$ — the outlier drags it up.
- Median of the same $= 100$ — the outlier is ignored entirely.
- Averaging *spreads* an impulse; median *deletes* it. Match filter to noise.

Math

The median minimizes $\sum_i |x_i - m|$ (robust); the mean minimizes $\sum_i (x_i - m)^2$, so a single far outlier dominates it.

Worked: median of the same patch

Same noisy patch as before:

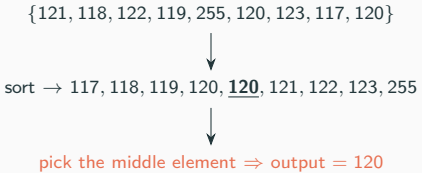
121	118	122
119	255	120
123	117	120

1. Sort the 9 values: $117, 118, 119, 120, \mathbf{120}, 121, 122, 123, 255$.
2. The 5th (middle) value is the **median** $= \mathbf{120}$.
3. The **255** sits harmlessly at the end of the list — it never reaches the middle.

✓ Good practice

Median gave **120** (the true gray); the mean gave **135**. Same data, right filter.

Median mechanics: sort & pick the middle



- It is **non-linear**: you cannot write it as a convolution kernel.
- It preserves **step edges** exactly (the middle value is a real pixel, never a blend).
- Cost: sorting each window; use small windows (3×3, 5×5) or a running histogram.

A quick reference of noise → filter

Noise	Looks like	Best filter
Gaussian	fuzzy grain everywhere	Gaussian / NLM
Salt-and-pepper	isolated black/white dots	median
Poisson (shot)	grain, worse in dark	NLM / variance-stabilize
Periodic	repeating wave/pattern	frequency-domain (Week 6)

Key idea

Periodic noise is invisible to spatial filters — you remove it in the frequency domain (Week 6).

The two core filters in code

```python
import cv2
gauss  = cv2.GaussianBlur(noisy, (5,5), 0)   # grain / Gaussian noise
median = cv2.medianBlur(noisy, 3)            # salt-and-pepper specks

# diagnose first: peek at a flat patch
patch = noisy[y:y+15, x:x+15]
print(patch.std())   # wide spread -> Gaussian; 0/255 spikes -> impulse
```

Note

Try median first for specks; averaging/Gaussian for grain. Measure, don't guess — Session 2 shows how (PSNR/SSIM).

Common pitfalls

✕ Wrong filter for the noise

Gaussian blur on salt-and-pepper just smears each speck into a gray blob — the dots survive.

✕ Even-sized / huge windows

Even windows have no single middle; oversized windows erase thin digit strokes. Use small, odd windows.

✕ Skipping the diagnosis

Applying a filter before looking at a flat patch wastes detail. Diagnose, then choose.

In-class exercise

In-class exercise (10 min)

Pick the filter — and defend it

For each situation, which filter and why?

1. A scanned meter with random black/white dots.
2. A low-light phone photo with fine grain everywhere.
3. A patch reads $\{119, 121, 0, 120, 255, 118, 122, 120, 121\}$ — compute both the mean and the median.
4. You have five still shots of the *same* meter. What is the cheapest way to cut the grain?

Debrief

- (1) Salt-and-pepper → **median**. (2) Gaussian grain → Gaussian/mean.
- (3) Mean $= \frac{996}{9} \approx 111$ (pulled by 0 and 255); median (sorted middle) $= 120$ — the true value.
- (4) **Average the five frames**: noise falls by $\sqrt{5} \approx 2.2\times$ with *zero* blur (more in Session 2).

Key idea

"Denoise" isn't one button — it's choosing the model that matches the noise.

Wrap-up

What you should now be able to do

- Name the physical origin of Gaussian, impulse, and Poisson noise.
- Diagnose noise from a flat patch and its histogram.
- Apply averaging/Gaussian for grain and median for specks — and justify each with a worked example.
- Reason quantitatively about the $\sqrt{N}$ noise reduction and the noise-vs-blur trade-off.

Next session

Session 2 — Edge-preserving restoration & measuring quality

- Filters that smooth flats *without* blurring edges: **bilateral** & **non-local means**.
- **Adaptive** & order-statistic filters; **temporal** multi-frame averaging.
- Deblurring & the PSF; and how to *measure* restoration with **PSNR**/**SSIM**.
- Then the week's deliverable. (No deliverable yet — just come with your worked exercise.)

Diagnose, then denoise.

Averaging for grain, median for specks — matched to the noise you actually see.